Operazioni asincrone e CompletableFuture in Java

Future

L'interfaccia Future<V> è utilizzata per rappresentare un risultato (di tipo V) che non è ancora disponibile ma lo sarà in un momento successivo, cioè nel futuro.

- **Ruolo**: rappresenta un compito (task) in esecuzione in modo asincrono e funge da riferimento al suo risultato finale.
- **Vantaggio**: l'uso di un Future consente al thread che ha avviato l'attività di continuare a svolgere altro lavoro invece di dover attendere immediatamente il completamento dell'operazione.
- **Creazione**: solitamente si utilizza un ExecutorService (thread creato da Executors e newCachedThreadPool) a cui si sottomette (tramite submit()) un oggetto Callable. Il metodo submit() ritorna immediatamente un oggetto Future mentre l'operazione viene eseguita su un thread separato.

Lettura del risultato da Future: get()

Per ottenere il valore concreto dall'oggetto Future, si utilizza il metodo get().

- **Comportamento bloccante**: se l'operazione asincrona non è ancora terminata quando si chiama get(), il thread chiamante si blocca e attende finché il risultato non diventa disponibile.
- **Comportamento immediato**: se l'operazione era già stata completata, get() restituisce immediatamente il valore.
- **Gestione del tempo** (timeout): esiste una versione overload del metodo get() che accetta un parametro di timeout. Questo definisce il tempo massimo di attesa. Se il risultato non arriva entro quel tempo, viene lanciata una TimeoutException.
- **Verifica**: è possibile usare il metodo isDone() per sapere, attraverso un valore booleano, se l'esecuzione è stata completata.

Considerazioni su Future

L'interfaccia Future presenta alcuni limiti e non permette di scrivere codice conciso per scenari complessi. In particolare, Future mostra carenze quando:

- si vuole fornire manualmente il risultato di un compito non ancora concluso (vedi dopo complete()).
- si vuole utilizzare il risultato di un'operazione asincrona come input per un'altra operazione asincrona, per concatenazione o *chaining*, (vedi dopo thenApply()).
- si ha la necessità di attendere il completamento di tutti i compiti in un elenco (vedi dopo allOf()).
- si vuole attendere il completamento solo del più veloce tra vari compiti (vedi dopo anyOf()).
- si vuole essere notificati del completamento tramite una callback per avviare altre attività, invece di dover bloccare il thread in attesa (vedi dopo callback()).

CompletableFuture supplyAsync()

Il metodo static supplyAsync() permette di avviare un'esecuzione asincrona (in parallelo) in modo conciso.

Prende in ingresso un Supplier<U> (un'interfaccia funzionale con un metodo get() che restituisce un valore) e avvia un thread separato (fornito dalgli Executors) per eseguirlo.

Restituisce, come valore di ritorno, immediatamente un nuovo oggetto CompletableFuture<U>, senza bloccare il thread chiamante, che rappresenterà il risultato futuro dell'operazione.

Esempio:

```
var getPrice = CompletableFuture.supplyAsync(() → p.computePrice("pen"));
```

CompletableFuture get()

Il metodo get() viene utilizzato per estrarre il valore concreto contenuto nel CompletableFuture.

Se il risultato è già pronto, lo restituisce immediatamente. Se l'operazione asincrona non è ancora finita, il thread che chiama get() si blocca e aspetta fino al completamento.

Può lanciare eccezioni se il thread viene interrotto (InterruptedException) o se l'esecuzione asincrona ha generato un errore (ExecutionException)

Esempio:

```
var getPrice = CompletableFuture.supplyAsync(() → p.computePrice("pen"));
//
try {
        System.out.println("Caller: price " + getPrice.get());
} catch (InterruptedException e) {
        e.printStackTrace();
} catch (ExecutionException e) {
        e.printStackTrace();
}
```

CompletableFuture complete()

Il metodo complete(T value) permette di terminare manualmente un CompletableFuture impostando forzatamente il suo risultato.

Se l'esecuzione non è ancora conclusa, questo metodo imposta il valore che verrà poi restituito dalle chiamate a get().

Ritorna *true* se il metodo è riuscito ad impostare il valore (e quindi a completare il future), oppure *false* se l'esecuzione era già stata completata in precedenza.

Esempio:

```
var getPrice = CompletableFuture.supplyAsync(()-> p.computePrice("Book"));
//
boolean fixed = getPrice.complete(20d);
if (fixed) System.out.println("Yes, fixed");
```

CompletableFuture con Timeout

Esistono varianti dei metodi per gestire i limiti di tempo ed evitare attese infinite:

- `get(long timeout, TimeUnit unit)`: aspetta il risultato solo per il tempo specificato. Se il tempo scade, lancia una TimeoutException.

```
getPrice.get(50L, TimeUnit.MILLISECONDS);
```

- `completeOnTimeout(T value, long timeout, TimeUnit unit)`: se l'operazione non finisce entro il timeout, completa automaticamente il future con un valore di default (value) specificato.

```
getPrice.completeOnTimeout(20d, 50L, TimeUnit.MILLISECONDS);
```

- `orTimeout(long timeout, TimeUnit unit)`: se l'operazione non finisce entro il timeout, la completa eccezionalmente lanciando un'eccezione.

```
getPrice.orTimeout(50L, TimeUnit.MILLISECONDS);
```

CompletableFuture thenApply()

Questo metodo viene utilizzato quando si vuole usare il risultato di una prima esecuzione asincrona come input per un'altra esecuzione asincrona.

La chiamata avviene in modo sincrono non appena il risultato della prima esecuzione diventa disponibile.

thenApply() prende in ingresso una funzione che lavora sul contenuto del primo CompletableFuture. Restituisce un nuovo CompletableFuture che conterrà il risultato delle due operazioni (quella originale e la trasformazione).

Inoltre, il metodo non blocca il codice chiamante.

Esempio:

```
var getPrice = CompletableFuture.supplyAsync(() -> p.computePrice("boo
k")); // computePrice(String) restituisce double
var getStrPrice = getPrice.thenApply(d -> p.transform(d)); // transform(do
uble) restituisce String
```

CompletableFuture allOf()

Si utilizza quando si vuole attendere il completamento di tutti i compiti svolti da un gruppo di esecuzioni asincrone.

È un metodo statico che accetta un array (o vari parametri) di CompletableFuture.

Ritorna un CompletableFuture<Void> che si considera completato solo quando tutti gli elementi dell'array hanno terminato la loro esecuzione. Solitamente si usa il metodo .join() (che ritorna un Void) su questo risultato per attendere effettivamente la fine di tutte le operazioni.

Esempio:

```
CompletableFuture<Double>[] prezzi = new CompletableFuture[2];
prezzi[0] = CompletableFuture.supplyAsync(() -> calcolaPrezzo("prodotto
```

```
0"));
prezzi[1] = CompletableFuture.supplyAsync(() -> calcolaPrezzo("prodotto
1"));
CompletableFuture.allOf(prezzi).join();
try {
    System.out.println(prezzi[0].get());
    System.out.println(prezzi[1].get());
} catch (InterruptedException e) {
    e.printStackTrace();
} catch (ExecutionException e) {
    e.printStackTrace();
}
```

CompletableFuture anyOf()

Si utilizza quando si vuole attendere il completamento solo del compito più veloce tra vari compiti asincroni, prelevandone il risultato.

È un metodo statico che restituisce un CompletableFuture completato non appena uno qualsiasi degli elementi dell'array termina.

Il CompletableFuture restituito contiene un Object (il risultato di chi ha finito prima). Se il compito più veloce completa con un'eccezione, allora anche il risultato di anyOf completerà con un'eccezione.

Esempio:

```
CompletableFuture<Double>[] prezzi = new CompletableFuture[2];
prezzi[0] = CompletableFuture.supplyAsync(() -> calcolaPrezzo("prodotto
0"));
prezzi[1] = CompletableFuture.supplyAsync(() -> calcolaPrezzo("prodotto
1"));
var p = CompletableFuture.anyOf(prezzi); // il tipo ritornato da anyOf() c
ontiene un Object
```

Callback thenApply() e thenAccept()

Questi metodi permettono di registrare delle callback, ovvero chiamate che avvengono al verificarsi di un evento (il completamento dell'attività asincrona).

→ **thenApply()**: registra una funzione che viene chiamata quando l'esecuzione precedente è completa e il valore di ritorno è disponibile (usato per trasformare i dati).

```
var getPrice = CompletableFuture.supplyAsync(() -> p.computePrice("boo
k")); // computePrice(String) restituisce double
var getStrPrice = getPrice.thenApply(d -> p.transform(d)); // transform(do
uble) restituisce String
```

→ **thenAccept()**: prende come parametro un Consumer (un metodo che "consuma" il valore). Restituisce un CompletableFuture<Void>. A differenza di thenApply, thenAccept non può contenere un risultato da passare avanti, ma indica solo se il compito è stato completato o meno.

```
prSconto.thenAccept(System.out::println);
```

Conclusione con eccezione

È possibile forzare il completamento di un CompletableFuture con un errore.

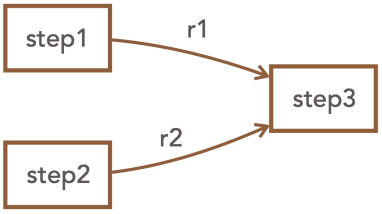
Si utilizza il metodo completeExceptionally(). Questo metodo fa sì che l'invocazione di get() lanci un'eccezione se l'esecuzione rappresentata dal CompletableFuture non era già stata conclusa.

Esempio:

```
CompletableFuture<String> cf = CompletableFuture.supplyAsync(() -> calc
());
cf.completeExceptionally(new RuntimeException("exceptionally"));
try {
    System.out.println("value: " + cf.get());
} catch (InterruptedException | ExecutionException e) {
    System.out.println("excep: " + e.getMessage());
}
```

Alcuni step di esecuzione asincrona

Spesso, per completare un lavoro (task o workflow), è necessario eseguire più passi (step) in sequenza o in parallelo. Supponiamo di avere due step indipendenti (step1 e step2). Questi possono essere avviati in thread dedicati, lavorando in parallelo, ciascuno con il proprio tempo di esecuzione. I risultati prodotti da questi due step (che chiamiamo r1 ed r2) possono essere entrambi utili o necessari a uno step successivo (step3), il quale deve aspettare l'esecuzione degli step precedenti per poter essere eseguito.

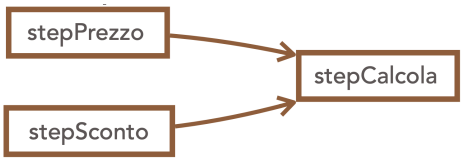


Esempio di esecuzione di step

Consideriamo uno scenario con tre operazioni:

→ **stepPrezzo()** e **stepSconto()**: due operazioni che restituiscono un double e possono essere avviate in parallelo.

→ **stepCalcola()**: un'operazione che prende un parametro double e restituisce un CompletableFuture<Double>.



Soluzione 1 (con blocco sul chiamante)

I primi due step vengono avviati in modo asincrono usando CompletableFuture.supplyAsync() (creando le variabili fpr ed fsc). Tuttavia, se per passare l'argomento a stepCalcola() si utilizza il metodo join() sui risultati parziali (es. stepCalcola(fpr.join() * fsc.join())), la chiamata blocca il chiamate. Questo accade perché join() impone un'attesa sincrona affinché i metodi stepSconto() e stepPrezzo() completino la loro esecuzione e restituiscano i valori.

```
var fsc = CompletableFuture.supplyAsync(() -> stepSconto());
var fpr = CompletableFuture.supplyAsync(() -> stepPrezzo());
// blocking wait
var d = stepCalcola(fpr.join() * fsc.join());
System.out.println("prezzo scontato: " + d.join());
```

Metodi per non bloccare il chiamante

Per evitare il blocco del chiamante, si utilizzano due metodi specifici:

→ **thenCombine()**: prende in input due parametri, un (altro) CompletableFuture e una funzione con due argomenti. Attende che sia il CompletableFuture su cui è invocato, sia quello passato come parametro abbiano completato l'esecuzione. A quel punto, passa i due risultati alla funzione fornita e restituisce un nuovo CompletableFuture contenente il risultato di questa funzione.

```
var fsc = CompletableFuture.supplyAsync(() -> stepSconto());
var fpr = CompletableFuture.supplyAsync(() -> stepPrezzo());
var fpp = fpr.thenCombine(fsc, (a, b) -> a * b);
```

→ **thenCompose()**: prende in input una funzione con un parametro e restituisce un CompletableFuture. Quando l'esecuzione del CompletableFuture su cui è chiamato termina, invoca la funzione passata e restituisce il CompletableFuture risultante da tale funzione.

```
var fps = fpp.thenCompose(p -> stepCalcola(p));
```

Esecuzione di step

Soluzione 2 (non bloccante)

Utilizzando i metodi appena descritti, è possibile implementare una seconda soluzione che non blocca il chiamante. Si avviano gli step preliminari in modo asincrono. Successivamente, si usa thenCombine() per aggregare i risultati dei primi due step (es. moltiplicandoli tra loro) e si concatena thenCompose() per indicare che stepCalcola() dipende da questi risultati. In questo modo si crea una catena fluida di chiamate in cui il chiamante principale non viene mai bloccato durante le attese.

```
var fsc = CompletableFuture.supplyAsync(() -> stepSconto());
var fps = CompletableFuture.supplyAsync(() -> stepPrezzo())
                            .thenCombine(fsc, (a, b) -> a * b)
                            .thenCompose(p -> stepCalcola(p));
System.out.println("prezzo scontato: " + fps.join());
```

Confronto fra metodi

Bisogna evidenziare una differenza fondamentale e il motivo per cui, nell'esempio, si usa thenCompose() invece di thenApply(). Sapendo che il metodo stepCalcola() restituisce già di per sé un oggetto di tipo CompletableFuture<Double>:

- se si usa la chiamata a **thenCompose()**, questa avvia l'operazione e restituisce un valore di ritorno "piatto" e corretto di tipo **CompletableFuture<Double>**.
- se si usasse invece **thenApply()**, si verificherebbe un annidamento indesiderato. Il valore di ritorno diventerebbe infatti un **CompletableFuture<CompletableFuture<Double>>**, rendendo la gestione del risultato finale molto più complessa.

Al seguente link è possibile vedere un esempio di utilizzo del CompletableFuture: https://github.com/isd-inf/request-batch-future.